МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Київський національний університет будівництва і архітектури
Кафедра кібербезпеки та комп`ютерної інженерії

ЗВІТ

з лабораторної роботи № 5

Реалізація запису та відтворення сценаріїв руху (Record / Playback)

Виконав: студент групи БІКСм-26
Тарасов Максим

Перевірив: Маринський К.В

Київ – 2026

1. Опис Record та Playback нод

Метою роботи було сформувати навички запису команд руху робота, їх збереження у файл та подальшого відтворення, а також зрозуміти принципи реалізації сценарної поведінки робототехнічної системи. Роботу виконано у віртуальній машині з Ubuntu та ROS 2 дистрибутива Lyrical, у робочому просторі ros2_ws, створеному в попередніх лабораторних роботах (використано готові Motion Node та GUI-ноду з роботи № 3).

Система складається з двох нод, об’єднаних в один пакет scenario_recorder, і спільного файлу сценарію scenario.csv:

- Record Node (record_node) підписується на топік /cmd_vel (тип geometry_msgs/msg/Twist), для кожного отриманого повідомлення обчислює час від початку запису та дописує до CSV-файлу рядок «час, linear_x, angular_z»;
- Playback Node (playback_node) зчитує CSV-файл, послідовно публікує збережені команди в /cmd_vel та перед кожною командою витримує паузу, що дорівнює різниці між часовими мітками сусідніх рядків; наприкінці публікується нульова команда зупинки.

Така архітектура не залежить від джерела команд: Record Node фіксує все, що з’являється в /cmd_vel (GUI, клавіатура, скрипт), а Playback Node подає ті самі команди назад, і Motion Node не відрізняє відтворений сценарій від ручного керування.

Спочатку в каталозі ~/ros2_ws/src створено пакет scenario_recorder типу ament_python (рисунок 1). Під час створення з’явилось попередження про невказану ліцензію (license TODO), яке не впливає на роботу пакета:

```
source /opt/ros/lyrical/setup.bash
source ~/ros2_ws/install/setup.bash
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python scenario_recorder \
    --dependencies rclpy geometry_msgs
```

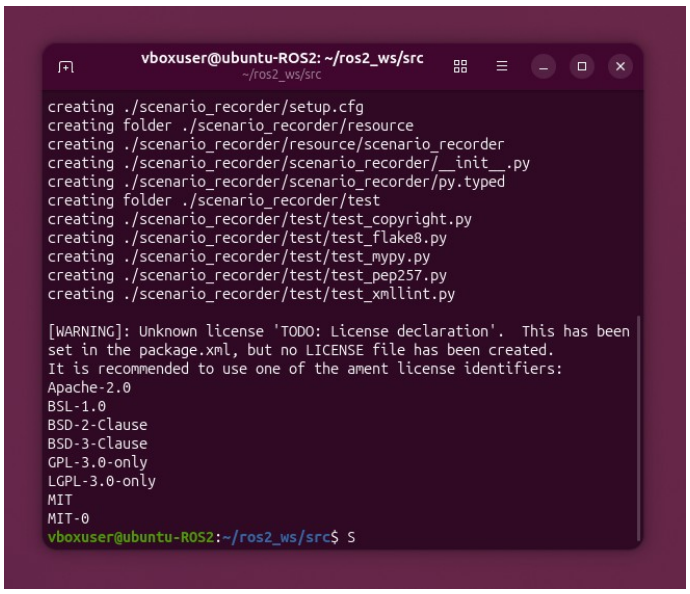


Рисунок 1 – Створення пакета scenario_recorder

Щоб ноди можна було запускати командою ros2 run, у файлі setup.py розділ entry_points доповнено двома точками входу: record_node і playback_node (рисунок 2):

```
entry_points={
    'console_scripts': [
        'record_node = scenario_recorder.record_node:main',
        'playback_node = scenario_recorder.playback_node:main',
    ],
},
```

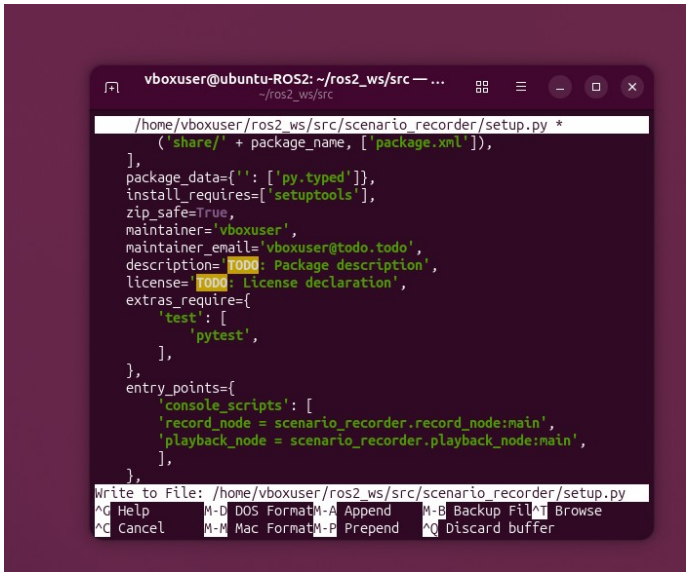


Рисунок 2 – Реєстрація точок входу record_node і playback_node у файлі setup.py

Пакет зібрано командою colcon build --packages-select scenario_recorder. Збірка завершилась успішно за 1.92 с (загальний час 2.53 с), після чого підключено середовище командою source install/setup.bash (рисунок 3).

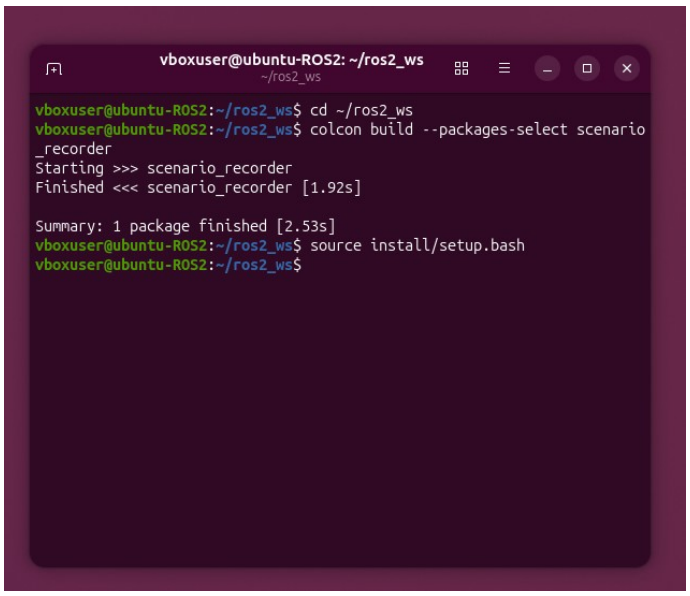


Рисунок 3 – Збірка пакета scenario_recorder

2. Фрагменти коду

2.1. Запис команд (Record Node)

У конструкторі ноди створюється підписка на /cmd_vel, визначається файл сценарію ~/ros2_ws/scenario.csv, фіксується момент початку запису start_time, а у файл записується заголовок. Файл відкривається в режимі "w", тому при кожному новому запуску записувача попередній сценарій перезаписується:

```
self.subscription = self.create_subscription(
    Twist, '/cmd_vel', self.cmd_callback, 10
)

self.filename = os.path.expanduser('~/ros2_ws/scenario.csv')
self.start_time = time.time()
self.recording = True

with open(self.filename, 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['time', 'linear_x', 'angular_z'])
```

Під час отримання кожного повідомлення обчислюється час від початку запису, зчитуються linear.x та angular.z, після чого рядок дописується до файлу (режим "a") з трьома знаками після коми:

```
def cmd_callback(self, msg):
    current_time = time.time() - self.start_time
    linear_x = msg.linear.x
    angular_z = msg.angular.z

    with open(self.filename, 'a', newline='') as f:
        writer = csv.writer(f)
        writer.writerow([f'{current_time:.3f}',
                         f'{linear_x:.3f}', f'{angular_z:.3f}'])
```

Запис зупиняється комбінацією Ctrl+C: виняток KeyboardInterrupt перехоплюється в main, нода виводить «Запис зупинено» і коректно завершується.

2.2. Зчитування сценарію (Playback Node)

Перед відтворенням нода перевіряє існування файлу, зчитує його за допомогою csv.DictReader у список словників і перевіряє, що файл не порожній:

```
if not os.path.exists(self.filename):
    self.get_logger().error(f'Файл не знайдено: {self.filename}')
    return

with open(self.filename, 'r') as f:
    reader = csv.DictReader(f)
    data = list(reader)

if not data:
    self.get_logger().warning('Файл порожній')
    return
```

2.3. Відтворення та синхронізація часу

Для кожного рядка визначається затримка як різниця між поточною та попередньою міткою часу; перед публікацією нода «засинає» на цей інтервал за допомогою time.sleep, що відтворює початкові паузи між командами. Після останньої команди публікується порожнє повідомлення Twist (усі швидкості нульові), яке зупиняє робота:

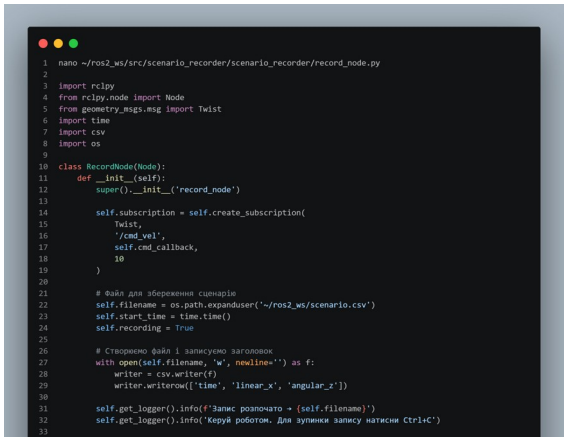
```
previous_time = 0.0
for row in data:
    current_time = float(row['time'])
    linear_x = float(row['linear_x'])
    angular_z = float(row['angular_z'])

    delay = current_time - previous_time
    if delay > 0:
        time.sleep(delay)

    msg = Twist()
    msg.linear.x = linear_x
    msg.angular.z = angular_z
    self.publisher_.publish(msg)
    previous_time = current_time

stop_msg = Twist()
self.publisher_.publish(stop_msg)
```

Повний код нод наведено на рисунках 4 і 5.



```
nano ~/ros2_ws/src/scenario_recorder/scenario_recorder/record_node.py

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
import time
import csv
import os

class RecordNode(Node):
    def __init__(self):
        super().__init__('record_node')

        self.subscription = self.create_subscription(
            Twist,
            '/cmd_vel',
            self.cmd_callback,
            10
        )

        # Файл для збереження сценарію
        self.filename = os.path.expanduser('~/ros2_ws/scenario.csv')
        self.start_time = time.time()
        self.recording = True

        # Створюємо файл і записуємо заголовок
        with open(self.filename, 'w', newline='') as f:
            writer = csv.writer(f)
            writer.writerow(['time', 'linear_x', 'angular_z'])

        self.get_logger().info(f'Запис розпочато → {self.filename}')
        self.get_logger().info('Керуй роботом. Для зупинки запису натисни Ctrl+C')
```



```
    def cmd_callback(self, msg):
        if not self.recording:
            return

        current_time = time.time() - self.start_time
        linear_x = msg.linear.x
        angular_z = msg.angular.z

        # Записуємо у файл
        with open(self.filename, 'a', newline='') as f:
            writer = csv.writer(f)
            writer.writerow([f'{current_time:.3f}', f'{linear_x:.3f}', f'{angular_z:.3f}'])

        self.get_logger().info(
            f'Записано: t={current_time:.2f}s | linear.x={linear_x:.2f} | angular.z={angular_z:.2f}'
        )

def main(args=None):
    rclpy.init(args=args)
    node = RecordNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info('Запис зупинено')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Рисунок 4 – Повний код Record Node (record_node.py)



```
nano ~/ros2_ws/src/scenario_recorder/scenario_recorder/playback_node.py

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
import csv
import time
import os

class PlaybackNode(Node):
    def __init__(self):
        super().__init__('playback_node')

        self.publisher_ = self.create_publisher(Twist, '/cmd_vel', 10)

        self.filename = os.path.expanduser('~/ros2_ws/scenario.csv')

        if not os.path.exists(self.filename):
            self.get_logger().error(f'Файл не знайдено: {self.filename}')
            return

        self.get_logger().info(f'Відтворення сценарію з файлу: {self.filename}')
        self.play_scenario()

    def play_scenario(self):
        with open(self.filename, 'r') as f:
            reader = csv.DictReader(f)
            data = list(reader)

        if not data:
            self.get_logger().warning('Файл порожній')
            return

        previous_time = 0.0
```

```
        for row in data:
            current_time = float(row['time'])
            linear_x = float(row['linear_x'])
            angular_z = float(row['angular_z'])

            # Затримка між командами (синхронізація часу)
            delay = current_time - previous_time
            if delay > 0:
                time.sleep(delay)

            # Публікуємо команду
            msg = Twist()
            msg.linear.x = linear_x
            msg.angular.z = angular_z
            self.publisher_.publish(msg)

            self.get_logger().info(
                f'Відтворено: t={current_time:.2f}s | linear.x={linear_x:.2f} | angular.z={angular_z:.2f}'
            )

            previous_time = current_time

        # Зупинка в кінці
        stop_msg = Twist()
        self.publisher_.publish(stop_msg)
        self.get_logger().info('Відтворення завершено')

def main(args=None):
    rclpy.init(args=args)
    node = PlaybackNode()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Рисунок 5 – Повний код Playback Node (playback_node.py)

3. Приклад файлу сценарію

Сценарій зберігається у текстовому файлі формату CSV згідно з Додатком 3: перший рядок містить заголовок, кожний наступний – одну команду. Вміст записаного файлу scenario.csv, виведений командою cat ~/ros2_ws/scenario.csv, наведено на рисунку 6 та в таблиці 1.

```
time,linear_x,angular_z
22.534,0.500,0.000
22.907,0.000,1.000
23.246,0.000,-1.000
23.537,-0.500,0.000
24.179,0.000,1.000
```

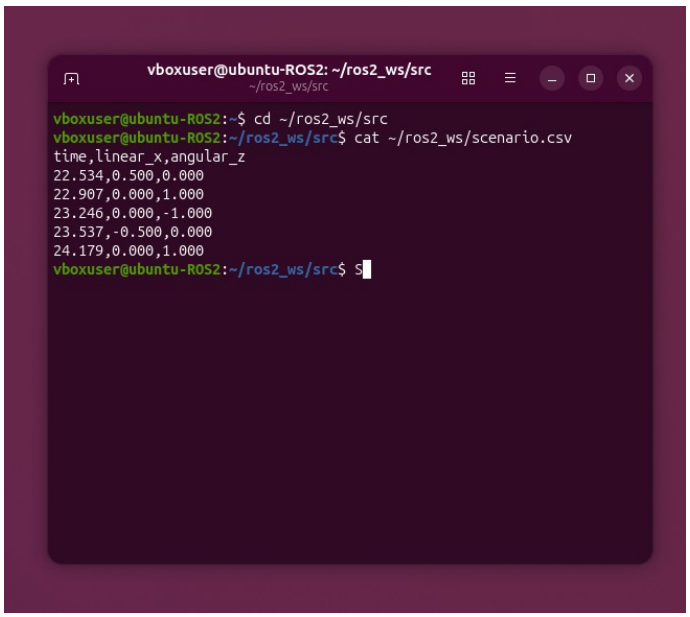


Рисунок 6 – Файл сценарію scenario.csv

Таблиця 1 – Зміст записаного сценарію

№	time, с	linear_x, м/с	angular_z, рад/с	Команда
1	22.534	0.500	0.000	Вперед
2	22.907	0.000	1.000	Ліворуч
3	23.246	0.000	-1.000	Праворуч
4	23.537	-0.500	0.000	Назад
5	24.179	0.000	1.000	Ліворуч

Час відраховується від моменту запуску Record Node, тому перша команда має мітку 22.534 с – стільки минуло між запуском запису та першим натисканням кнопки.

4. Результати виконання

4.1. Запис сценарію

Для запису в окремих терміналах запущено Motion Node (ros2 run motion_control motion_node), GUI-ноду керування (ros2 run robot_gui gui_node) та Record Node (ros2 run scenario_recorder record_node). Record Node вивела повідомлення про початок запису у файл /home/vboxuser/ros2_ws/scenario.csv, Motion Node перейшла в режим очікування команд, а вікно GUI показало статус «Очікування» (рисунок 7).

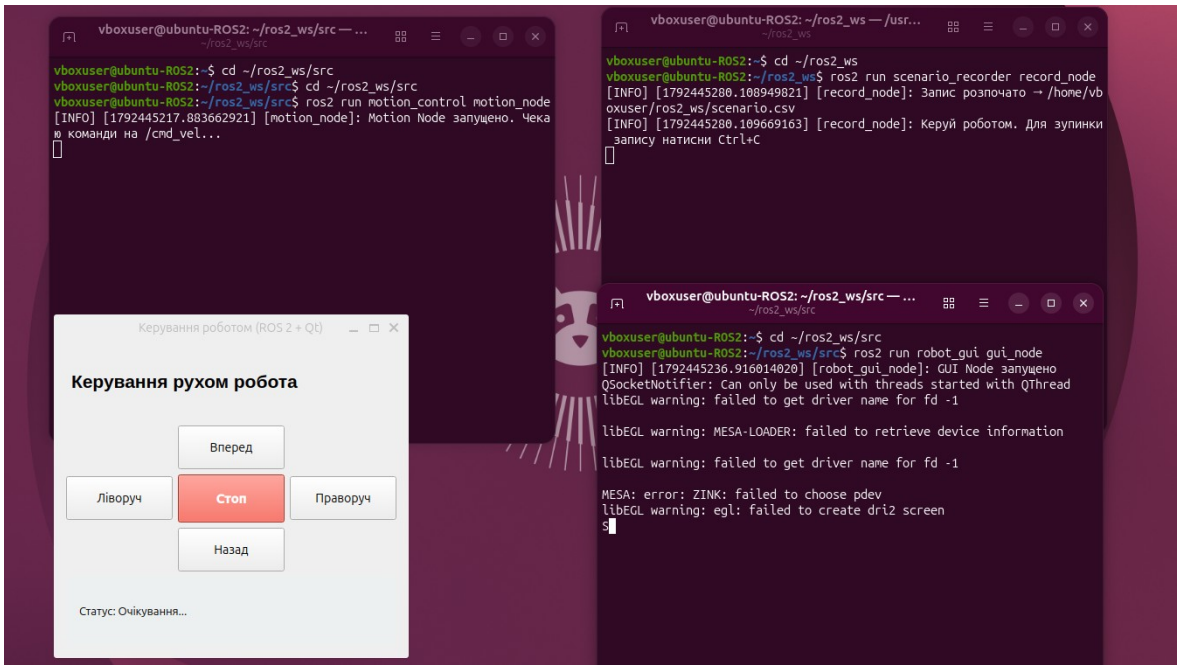


Рисунок 7 – Запуск Motion Node, GUI-ноди та Record Node

Далі ручним керуванням через GUI виконано послідовність із п’яти команд: «Вперед», «Ліворуч», «Праворуч», «Назад», «Ліворуч». Кожну команду отримали й Motion Node («Отримано: linear.x=…, angular.z=…»), і Record Node («Записано: t=… | linear.x=… | angular.z=…»); значення в журналах обох нод збігаються з командами GUI (рисунок 8).

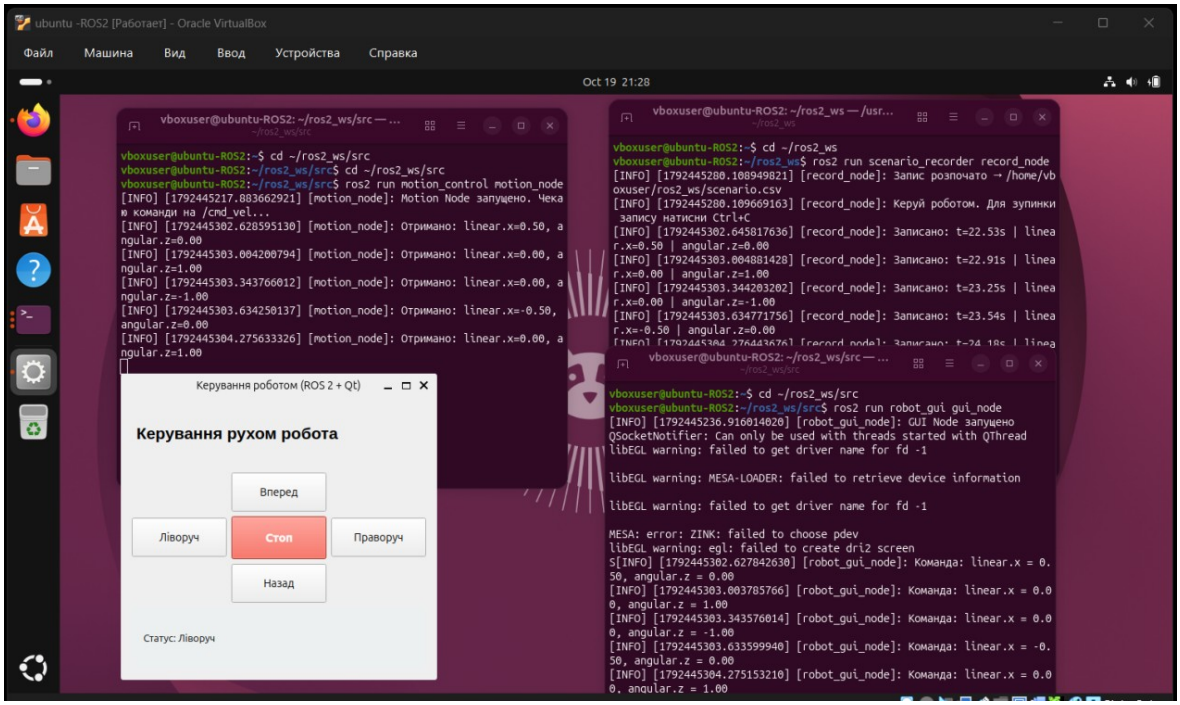


Рисунок 8 – Запис сценарію під час ручного керування через GUI

4.2. Відтворення сценарію

Після зупинки запису (Ctrl+C) запущено Motion Node та Playback Node (ros2 run scenario_recorder playback_node). Playback Node прочитала файл scenario.csv, послідовно опублікувала всі п’ять команд і завершила роботу повідомленням «Відтворення завершено». Motion Node отримала ті самі команди й додатково нульову команду зупинки (linear.x=0.00, angular.z=0.00) наприкінці (рисунок 9).

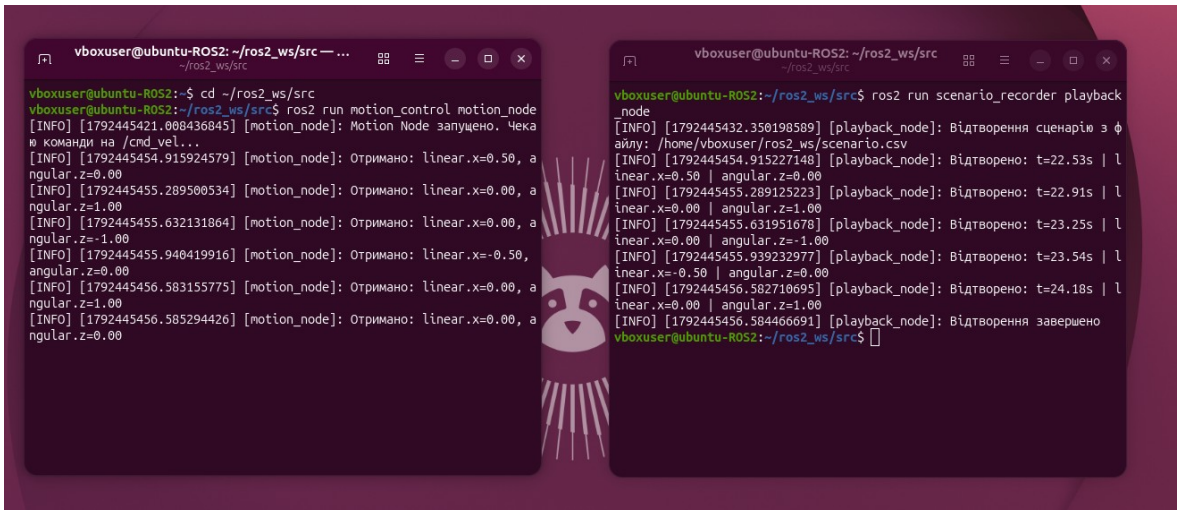


Рисунок 9 – Відтворення сценарію: Motion Node (ліворуч) і Playback Node (праворуч)

4.3. Перевірка відтворення

Повторюваність руху та збереження послідовності підтверджуються збігом значень: п’ять команд, відтворених Playback Node, і п’ять команд, отриманих Motion Node, відповідають записаному сценарію в тому самому порядку (вперед → ліворуч → праворуч → назад → ліворуч). Коректність затримок перевірено порівнянням інтервалів між командами в записі та у відтворенні за часовими мітками журналів (таблиця 2).

Таблиця 2 – Порівняння інтервалів між командами при записі та відтворенні

Перехід	**Інтервал у записі, с**	**Інтервал у відтворенні, с**	**Різниця, с**
1 → 2	0.373	0.374	0.001
2 → 3	0.339	0.343	0.004
3 → 4	0.291	0.307	0.016
4 → 5	0.642	0.643	0.001

Інтервали відтворення відрізняються від записаних не більше ніж на 0.016 с, що для ручного керування кнопками є несуттєвим. Невелике збільшення інтервалів пов’язане з тим, що time.sleep гарантує лише мінімальну тривалість паузи, а також з часом, який витрачається на виконання самого циклу (відкриття логера, публікація повідомлення).

Окремо слід відзначити першу паузу: оскільки previous_time ініціалізується нулем, перед першою командою нода чекає 22.534 с, тобто відтворює і «мовчання» на початку запису (від запуску Playback Node до першої команди пройшло близько 22.6 с). Якщо ця пауза небажана, достатньо ініціалізувати previous_time значенням часу першого рядка.

5. Висновки

У ході лабораторної роботи створено пакет scenario_recorder на Python (ament_python) із двома нодами: Record Node, яка підписується на /cmd_vel і записує час, linear.x та angular.z у файл scenario.csv, та Playback Node, яка зчитує цей файл і відтворює команди в /cmd_vel з тими самими затримками. Для синхронізації часу використано різницю міток часу сусідніх рядків та time.sleep.

Експеримент показав, що записаний за допомогою GUI сценарій із п’яти команд відтворюється без втрат: команди повторюються в тій самій послідовності, а відхилення інтервалів не перевищують 0.016 с. Наприкінці відтворення робот отримує нульову команду й зупиняється, що убезпечує систему від «залипання» останньої команди.

Реалізований підхід дає основу для автоматизації поведінки робота: один раз записаний сценарій можна зберігати, редагувати й повторно запускати без участі оператора. Можливі вдосконалення: параметризація імені файлу через параметри ROS 2, ігнорування початкової паузи, відтворення таймером замість блокуючого sleep та запис у зручнішому для обробки форматі, наприклад rosbag.